Software Architecture Project

Social Networking Platform

Requirements Specification Document

Phase I

Team Members

- Khalid Mohamed 202300741
- Youssef Islam 202300886
- Abdelrahman Mohamed 202300056
- Amrou Mahmoud 202300491

GitHub Repository

- https://github.com/khalidua/Social-Networking-Platform.git

Video Presentation

- https://drive.google.com/file/d/1iwcqHyQaC07xTiBMRr08ledjbAw9oNe0/view?usp=sharing

Software Overview

A place where people connect, share posts, and follow each other, a lightweight facebook rip-off.

Software Aspects

Under the hood, it's built as a set of independent services:

- Auth — sign in with Google, get a token
- Users — profiles and follow relationships
- Posts — create, edit, delete posts
- Feed — your personalized homepage
- Notifications — alerts when someone follows or interacts with you

Each service runs independently, has its own database, and communicates either directly (sync) or through a message broker (async) depending on whether an immediate response is needed.

Requirements

1. Functional Requirements

Authentication

- User can sign in with Google OAuth2

- User can sign out
- System issues and validates JWT tokens

User Management

- User has a profile (name, bio, profile picture)
- User can follow / unfollow other users
- User can view other profiles

Posts (Core CRUD)

- User can create a post
- User can read/view posts
- User can update their own post
- User can delete their own post

Feed (Dashboard)

- User sees a personalized feed of posts from followed users
- Feed updates in real time or near real time

Notifications

- User receives a notification when followed
- User receives a notification when someone interacts with their post

2. Non-Functional Requirements & Architectural Drivers

Architectural Driver	Non-Functional Requirement	Measurable Target
Many users simultaneously	Scalability	*System handles 1,000 concurrent users without degradation*
Service failures shouldn't cascade	Resilience	*Core features remain available if any single service fails*
User data must be protected	Security	*Zero unauthenticated endpoints; max 100 requests/min per user*
Debugging must be fast	Observability	*All logs centralized; any issue traceable within 2 minutes*
Feed must feel instant	Performance	*Feed response time under 500ms at normal load*
Transient failures must self-heal	Fault tolerance	*Failed inter-service calls retry up to 3 times before circuit*

		opens

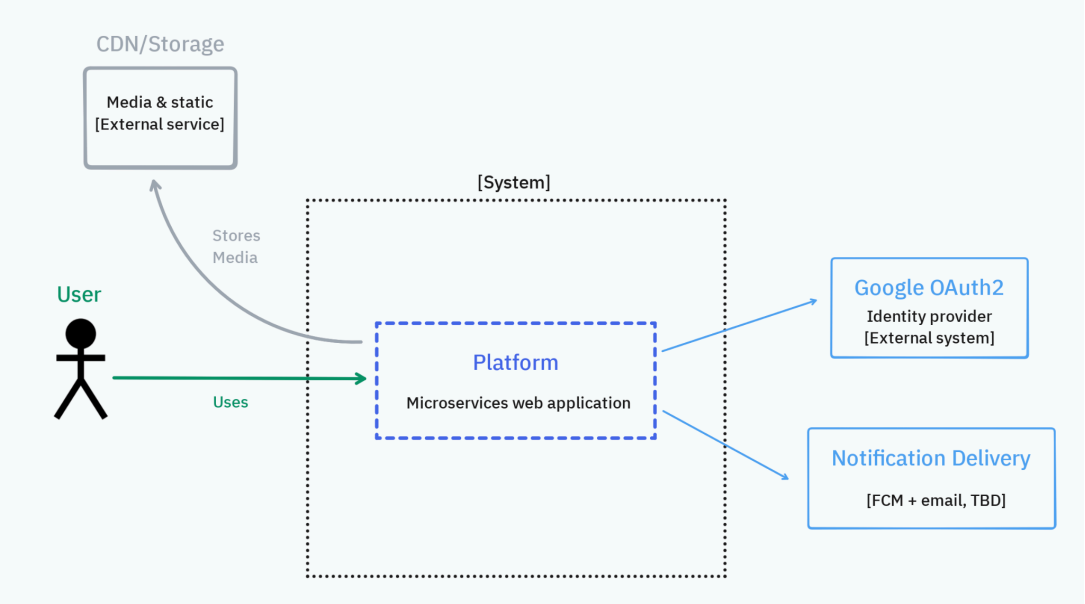
User Stories

1. ### Authentication
    a. As a user, I want to sign in with my Google account, so that I don't need to create a separate username and password.
    b. As a user, I want to sign out of my account, so that my account stays secure on shared devices.
2. ### Profile
    a. As a user, I want to view my profile, so that I can see how others see me.
    b. As a user, I want to edit my profile information, so that I can keep it up to date.
    c. As a user, I want to view other users' profiles, so that I can learn about them before following.
3. ### Posts
    a. As a user, I want to create a post, so that I can share content with my followers.
    b. As a user, I want to edit my post, so that I can correct mistakes after publishing.
    c. As a user, I want to delete my post, so that I can remove content I no longer want shared.
    d. As a user, I want to view a post, so that I can read content from people I follow.
4. ### Social
    a. As a user, I want to follow another user, so that their posts appear in my feed.
    b. As a user, I want to unfollow a user, so that I can stop seeing their content.
5. ### Feed
    a. As a user, I want to see a personalized feed of posts from people I follow, so that I can stay up to date with their content.
    b. As a user, I want my feed to update in real time, so that I don't miss new posts without refreshing.
6. ### Notifications
    a. As a user, I want to receive a notification when someone follows me, so that I know who is interested in my content.
    b. As a user, I want to receive a notification when someone interacts with my post, so that I can engage back with them.

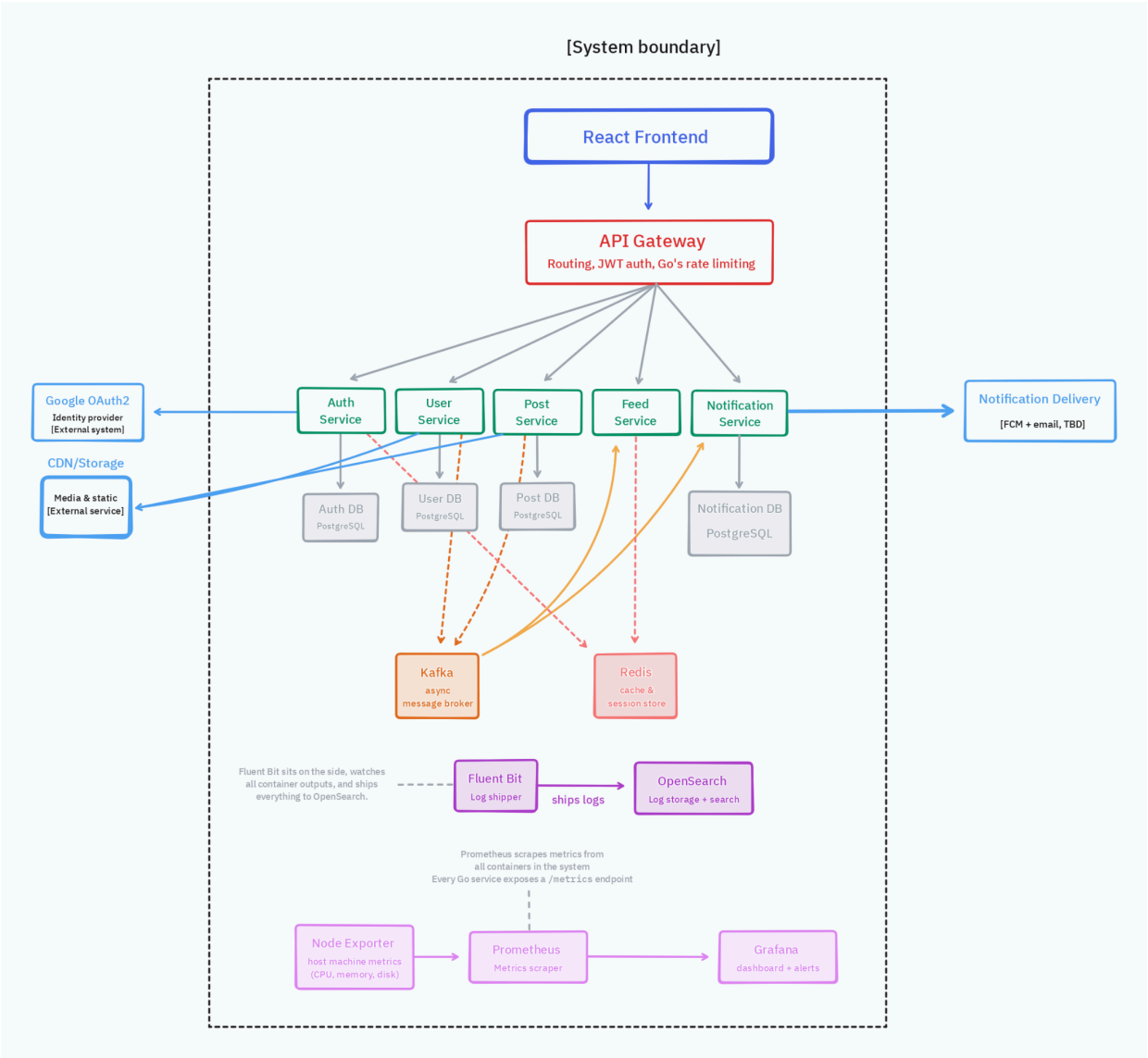
C4 Model Diagrams

1. ### Context Diagram

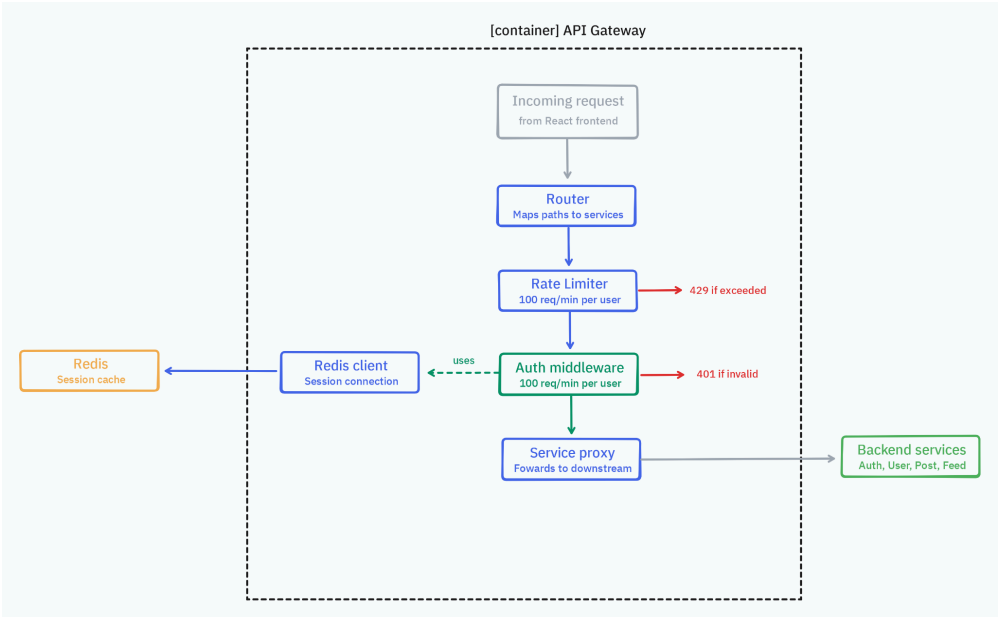
- https://www.tldraw.com/f/tmuZH3yiDattrGF5V6wJJ?d=v0.-83.1660.1114.page



2. Container Diagram
   - https://www.tldraw.com/f/tmuZH3yiDattrGF5V6wJJ?d=v-241.-55.2028.1361.cI5Z2HGq5Jbp5m6TNJVBL



3. Component Diagram
    a. API Gateway
    - https://www.tldraw.com/f/tmuZH3yiDattrGF5V6wJJ?d=v-343.-146.2361.1584.hQIxGmQhRueOH0I0Omrom



Architecture Decision Record

ADR-001: Microservices Architecture

- Date: 2026-03-27
- Status: Accepted
- Context:
  - The system has five business capabilities with different scaling and failure needs. A decision is needed on whether to build as a monolith or separate services.
- Decision:
  - Build five independently deployable microservices, each owning a single capability and its own database.
- Consequences:
  - Pros:
    - Services scale, deploy, and update independently
    - Failure in one service does not affect others
    - Teams can work on services in parallel
  - Cons:
    - Higher operational complexity
    - Network overhead between services
    - Debugging across service boundaries is harder

ADR-002: Apache Kafka as the Message Broker

- Date: 2026-03-27
- Status: Accepted

- Context:
    - Post creation and follow actions trigger side effects in other services. These should not block the original request or create tight service dependencies.
- Decision:
    - Use Kafka for all async communication. Post Service publishes post.created, User Service publishes user.followed. Feed Service and Notification Service consume independently.
- Consequences:
    - Pros:
        - Services are fully decoupled from each other
        - Messages persist to disk, consumers can replay missed events after recovery
        - Scales well as consumers grow
    - Cons:
        - Kafka requires its own container and configuration
        - Updates to feed and notifications are not instant
        - More complex than a simple HTTP call between services

ADR-003: Database per Service

- Date: 2026-03-27
- Status: Accepted
- Context:
    - A decision is needed on whether services share a database or each own a dedicated one. A shared database risks coupling services at the data layer.
- Decision:
    - Each service owns a dedicated PostgreSQL instance. No service queries another service's database. Cross-service data access happens only through API calls or Kafka events.
- Consequences:
    - Pros:
        - Schema changes in one service cannot break another
        - Each database scales and fails independently
        - True service isolation at the data layer
    - Cons:
        - More database instances to manage
        - No cross-service JOIN queries
        - Slight data duplication may occur

ADR-004: Feed Service Fan-out-on-write and No Persistent Database

- Date: 2026-03-27
- Status: Accepted
- Context:

        - The feed must respond under 500ms. Assembling a feed at read time by querying posts across all followed users is too slow. A decision is needed on how feed data is built and stored.
    - Decision:
        - When a post.created event arrives from Kafka, the Feed Service pre-writes the post into each follower's feed in Redis. Feed reads are a direct Redis lookup. No persistent database is used.
    - Consequences:
        - Pros:
            - Feed reads are a single Redis lookup regardless of follow count
            - Directly meets the 500ms NFR
            - No database to manage for the Feed Service
        - Cons:
            - One post triggers one Redis write per follower
            - Feed data is lost on Redis restart
            - Does not scale well for users with very large follower counts

ADR-005: Google OAuth2 and JWT for Authentication

- Date: 2026-03-27
- Status: Accepted
- Context:
    - The system must authenticate users without managing passwords. User identity must be verifiable on every request without hitting a database each time.
- Decision:
    - Users sign in via Google OAuth2. Auth Service issues a signed JWT and stores the session in Redis. The API Gateway validates the JWT against Redis on every request.
- Consequences:
    - Pros:
        - System never stores passwords
        - JWT validation is one Redis read per request
        - Google handles credential security on behalf of the platform
    - Cons:
        - New logins fail if Google is unreachable
        - Users without a Google account cannot register
        - Stolen tokens remain valid until the Redis session expires

ADR-006: Redis for Caching and Session Storage

- Date: 2026-03-27
- Status: Accepted
- Context:
    - JWT validation happens on every request and feed assembly is costly. Both are too expensive to run against a database each time.

- Decision:
    - Redis serves two purposes. Sessions stored under session:{user_id} by Auth Service. Feed cache stored under feed:{user_id} by Feed Service. API Gateway reads sessions from Redis on every request.
- Consequences:
    - Pros:
        - JWT validation costs one Redis read per request
        - Feed reads served from memory, meets 500ms NFR
        - One instance serves both purposes
    - Cons:
        - Restart forces all users to re-login and feeds to rebuild
        - Auth Service, Feed Service, and API Gateway all depend on Redis availability
        - At higher scale, session and feed cache should be split into separate instances

API Specifications

RESTful API endpoints documented using OpenAPI (Swagger)

- https://github.com/khalidua/Social-Networking-Platform/tree/main/docs

Synchronous and asynchronous communication clearly explained

- https://github.com/khalidua/Social-Networking-Platform/tree/main/docs

Resilience Strategies

To ensure that the system remains stable and available even when some services fail, several resilience mechanisms are implemented. Since the platform is built using a microservices architecture, failures in one service must not cause the entire system to stop working. The system uses retries, circuit breakers, and fallbacks to prevent cascading failures and to maintain core functionality

5.1 Retries

Retries are used when a service call fails due to a temporary error such as network delay, timeout, or service overload.
When a service makes a synchronous request to another service, the request is retried up to **3 times** before reporting a failure.

Example:

- Feed service requests posts from Posts service
- If the request fails, it retries up to 3 times
- If all retries fail, fallback logic is used

Retries are implemented with exponential backoff to avoid overloading the system.

Justification:
Retries help recover from temporary failures without affecting the user experience and support the non-functional requirement of fault tolerance.

5.2 Circuit Breaker

Circuit breaker is used to stop repeated calls to a failing service.
If a service fails multiple times, the circuit breaker opens and prevents further calls for a short period.

Behavior:

- Closed → normal operation
- Open → calls blocked after repeated failures
- Half-open → test requests allowed to check recovery

Example:

- If Notifications service is down,
- Feed service stops calling it after multiple failures
- The system continues working without notifications

Justification:
Circuit breaker prevents cascading failures and keeps the system responsive even when one service is unavailable.
This supports the architectural driver:
"Service failures shouldn't cascade."

5.3 Fallback Mechanisms

Fallback is used when a service cannot respond.
Instead of failing completely, the system returns a default or partial response.

Examples:

Feed fallback:

- If Feed service cannot get fresh posts,
- Return cached feed data

Notification fallback:

- If Notifications service fails,
- Store events in message queue and send later

Auth fallback:

- If Google OAuth fails,
- User receives error but system stays operational

Justification:
Fallback ensures that core features remain available even when some services fail.
This satisfies the resilience and availability requirements.

5.4 Asynchronous Messaging for Fault Isolation

Some operations use a message broker instead of direct calls.

Used for:

- Notifications
- Feed updates
- Follow events

If the consumer service is down:

- Messages stay in queue
- Service processes them later

Justification:
Async communication reduces dependency between services and increases fault tolerance.

5.5 Rate Limiting and Protection

Each service limits the number of requests per user.

Rule:

- Max 100 requests per minute per user

Purpose:

- Prevent overload

- Protect against abuse
- Keep system stable